



Gymnázium J. K. Tyla

**DarkFactory:
Agentic Engineering in practice
(Agentické inženýrství v praxi)**

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Rozsah práce: 4863 slov / 32210 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Odborná práce zkoumá využití agentní umělé inteligence při vývoji softwaru se zaměřením na architekturu agentního harnessu. Teoretická část vymezuje mechanismy softwarového inženýrství, jazykových modelů, správy kontextu, nástrojů, řízení provádění a multiagentních systémů. Praktická část popisuje systém DarkFactory, který odděluje modelové rozhodování od trvalého stavu, prostředí, capabilities, GitHub control plane, ověřování a bezpečnostních hranic. Evaluace používá zdrojový kód, automatické testy a CI výsledky a rozlišuje implementované mechanismy od vlastností, které ještě vyžadují živý end-to-end důkaz.

Abstract

This thesis examines the use of agentic artificial intelligence in software development, focusing on the architecture of an agent harness. The theoretical part defines mechanisms from software engineering, language models, context management, tools, execution control, and multi-agent systems. The practical part describes DarkFactory, a system that separates model decisions from persistent state, the execution environment, capabilities, the GitHub control plane, verification, and security boundaries. The evaluation uses source code, automated tests, and CI results and distinguishes implemented mechanisms from properties that still require live end-to-end evidence.

Keywords (Klíčová slova)

Agentní sezení (Session), Agentní smyčka (Agent Loop), Cílené smyčky (Goal Loops), Degradace kontextu (Context Rot), Deterministické ověření (Deterministic Verification), Dovednosti (Skills), Finální kontrola souladu (Final Alignment), Guardrail, Harness, Hooks, Inferenční engine (Inference Engine), Integrovaný test (Integration Test), Izolované prostředí (Sandbox), Kompakce kontextu (Context Compaction), Kontextové inženýrství (Context Engineering), Kontextové okno (Context Window), MCP, Mezipaměť klíčů a hodnot (KV Cache), Nástroje (Tools), Orchestrátor (Orchestrator), Pluginy (Plugins), Plán DarkFactory (Planning), Plánování (Planning), Požadavek (Request), Pracovní graf (Workflow Graph), Prompt Injection, Promptové inženýrství (Prompt Engineering), Prostředí agenta (Agent Environment), Průběžná integrace (CI), Pull Request, Předání řízení (Handoff), Přepis (Transcript), RAG, Rekonciliace stavu (Reconciliation), Skripty (Scripts), Slop, Smyčka revize a opravy (Review/Fix Loop), Spouštění kódu (Code Execution), Správa verzí (Version Control), Stav (State), Subagent, Systémový prompt (System Prompt), Token, Tokenizér (Tokenizer), Transformer, Vektorová reprezentace (Embedding), Velký jazykový model (LLM), Vibe Coding, Vkládání kontextu (Context Injection), Vyvolávání nástrojů (Tool Calling), Vývoj řízený specifikací (Spec-Driven Development), Větev (Branch), Člověk ve smyčce (HITL)

Obsah

Prohlášení	2
Anotace	3
Abstract	4
Keywords (Klíčová slova)	5
1 Úvod	10
1.1 Motivace a vymezení problému	10
1.2 Východisko a argument práce	10
1.3 Cíle práce	10
1.3.1 Hlavní cíl	11
1.3.2 Dílčí cíle	11
1.4 Výzkumné otázky	11
1.5 Metodika práce	12
2 Teoretická část: Agentní vývoj softwaru	13
2.1 Úvod	13
2.2 AI-asistovaný vývoj	13
2.2.1 Úvod	13
2.2.2 Zadání a způsob práce	13
Vibe Coding	13
Vývoj řízený specifikací (Spec-Driven Development)	14
Plánování (Planning)	14
2.2.3 Řízení změny	14
Správa verzí (Version Control)	14
Větev (Branch)	15
Pull Request	15
2.2.4 Kvalita a ověřování	15
Slop	15
Průběžná integrace (CI)	15
Integrační test (Integration Test)	15
2.2.5 Závěr	15
2.3 Jazykový model a inference	15
2.3.1 Úvod	15
2.3.2 Jazykový model	16
Transformer	16
Velký jazykový model (LLM)	16

Tokenizér (Tokenizer)	16
Token	16
Vektorová reprezentace (Embedding)	16
2.3.3 Inference	17
Inferenční engine (Inference Engine)	17
Kontextové okno (Context Window)	17
Mezipaměť klíčů a hodnot (KV Cache)	17
2.3.4 Limity inference	18
Degradace kontextu (Context Rot)	18
Divergence modelu	18
2.3.5 Závěr	18
2.4 Harness	18
2.4.1 Úvod	18
2.4.2 Smyčka a stav	18
Agentní smyčka (Agent Loop)	18
Agentní sezení (Session)	19
Přepis (Transcript)	19
Stav (State)	19
2.4.3 Nástroje a prostředí	19
Prostředí agenta (Agent Environment)	19
Nástroje (Tools)	20
Vyvolávání nástrojů (Tool Calling)	20
Spouštění kódu (Code Execution)	20
Izolované prostředí (Sandbox)	20
2.4.4 Dovednosti a rozšíření	20
Dovednosti (Skills)	20
Pluginy (Plugins)	21
Skripty (Scripts)	21
Hooks	22
MCP	22
2.4.5 Závěr	22
2.5 Agentické inženýrství	22
2.5.1 Úvod	22
2.5.2 Instrukce a kontext	22
Promptové inženýrství (Prompt Engineering)	22
Systémový prompt (System Prompt)	22

Kontextové inženýrství (Context Engineering)	23
Vkládání kontextu (Context Injection)	23
Kompakce kontextu (Context Compaction)	23
RAG	23
Prompt Injection	23
2.5.3 Řízení agentního chování	23
Cílené smyčky (Goal Loops)	23
Guardrail	24
Člověk ve smyčce (HITL)	24
2.5.4 Orchestrace agentů	24
Subagent	24
Orchestrátor (Orchestrator)	24
Předání řízení (Handoff)	24
Pracovní graf (Workflow Graph)	25
2.5.5 Závěr	25
3 Praktická část	26
3.1 Úvod	26
3.2 DarkFactory	26
3.3 Životní cyklus změny	26
3.3.1 Úvod	26
3.3.2 Zadání a plán	26
Požadavek (Request)	26
Plán DarkFactory (Planning)	26
3.3.3 Implementace	27
3.3.4 Ověření a revize	27
Deterministické ověření (Deterministic Verification)	27
Smyčka revize a opravy (Review/Fix Loop)	27
3.3.5 Finalizace	27
Finální kontrola souladu (Final Alignment)	27
Rekonciliace stavu (Reconciliation)	28
3.3.6 Přerušování a obnova	28
3.3.7 Závěr	28
3.4 Vyhodnocení	28
3.4.1 Úvod	28
3.4.2 Ověření mechanismů	28
3.4.3 Ověření systému	29

3.4.4	Ověření na repozitářích	29
3.4.5	Výzkumné otázky	29
3.4.6	Diskuse a omezení	30
3.4.7	Závěr	30
4	Závěr	31
	Seznam obrázků a tabulek	32
	Seznam příloh	33
A	Encyklopedie a rejstřík pojmů	34
	Seznam zdrojů	38

1 Úvod

Tato práce zkoumá, jak lze současné agentní systémy používat při vývoji softwaru tak, aby jejich autonomie byla spojena s trvalým stavem, ověřitelnými účinky a explicitními kontrolními body.

Teoretická část postupuje od principů softwarového inženýrství přes vlastnosti a limity jazykového modelu k agentnímu harnessu a technikám Agentic Engineering. Praktická část tyto mechanismy vztahuje k systému DarkFactory a odděluje popis implementované architektury od výsledků, které lze doložit testy, CI a provozními artefakty.

Práce se nezaměřuje na trénování modelů. Jazykový model je chápán jako inferenční komponenta uvnitř širšího systému, jehož spolehlivost závisí také na správě stavu, prostředí, nástrojích, verifikaci, bezpečnostních hranicích a způsobu orchestrace.

1.1 Motivace a vymezení problému

Současné agentní systémy dokážou nad softwarovým projektem provádět více kroků, pracovat se soubory a nástroji a ověřovat vlastní změny. Hlavním problémem této práce proto není samotné generování kódu, ale technická vrstva, která převádí schopnosti jazykového modelu do řízeného, obnovitelného a ověřitelného vývojového procesu.⁽¹⁾

Současné agentní architektury propojují model s nástroji, pracovním prostředím a dlouhodobějším stavem, takže mohou provádět více navazujících kroků místo izolované textové odpovědi.^(1,2)

Rozšíření generativní AI zároveň vytváří praktický důvod tyto systémy studovat: Microsoft AI Economy Institute pro první čtvrtletí roku 2026 odhadl používání generativní AI na 17,8 % světové populace ve věku 15–64 let.⁽³⁾ Tento údaj v práci slouží pouze jako kontext rozšíření technologie, nikoli jako metrika kvality nebo autonomie agentních systémů. DarkFactory slouží jako konkrétní artefakt, na kterém práce zkoumá oddělení modelu, trvalého stavu, prostředí, nástrojů a kontrolních mechanismů.

1.2 Východisko a argument práce

Rostoucí schopnosti jazykových modelů mění vývoj softwaru od dílčí asistence k delegování stále delších a samostatnějších pracovních úloh.

Argument práce postupuje v jedné návaznosti: jak AI mění způsob zadávání, provádění a ověřování softwarové práce; co samotný model skutečně je a dokáže; co kolem něj přidává Harness; jak Agentic inženýrství tyto mechanismy záměrně skládá a omezuje; jak je realizuje DarkFactory; a co o výsledku skutečně ukazují dostupné důkazy.

1.3 Cíle práce

Tato část stanovuje hlavní a dílčí cíle práce.

Cíle vymezují, jaký artefakt má být navržen a které jeho vlastnosti mají být technicky ověřeny. Výzkumné otázky jsou odděleny do následující samostatné části.

1.3.1 Hlavní cíl

Navrhnout a implementovat architekturu agentního harnessu pro dlouhotrvající vývoj softwaru, která odděluje jazykový model od trvalého stavu, prostředí a deterministických kontrolních mechanismů, a technicky ověřit vlastnosti této architektury na systému DarkFactory.

DarkFactory je implementační artefakt této architektury. Splnění cíle se posuzuje podle dohledatelné implementace navržených mechanismů a reprodukovatelných testovacích nebo provozních důkazů. Úplný živý Request lifecycle je samostatná úroveň ověření a nesmí být zaměněn za samotnou existenci architektury nebo úspěšné komponentové testy.

1.3.2 Dílčí cíle

- Vymezit teoretické mechanismy Software Engineering, Modelu, Harnessu a Agentic Engineering potřebné pro dlouhotrvající agentní vývoj softwaru.
- Navrhnout a implementovat DarkFactory s explicitním stavem běhu, odděleným prostředím, capability rozhraním, GitHub control plane a oddělenými hranicemi lidské a strojové identity.
- Implementovat mechanismy řízeného životního cyklu požadavku: Planning, deterministické ověření, review/fix, Final Alignment, integraci a obnovu přerušného běhu.
- Ověřit implementované mechanismy automatickými testy a CI nad konkrétním commitem.
- Ověřit přenositelnost vybraných částí řešení na konkrétních cílových repozitářích a samostatně vyhodnotit, zda existuje důkaz celého živého end-to-end Request lifecycle.
- Vztáhnout zjištěné výsledky a jejich omezení přímo k výzkumným otázkám.

Dílčí cíle oddělují teoretické vymezení, konstrukci artefaktu a jednotlivé úrovně jeho ověření. Nesplněná nebo neprokázaná úroveň evaluace proto nemusí být nahrazena silnějším tvrzením z jiné vrstvy důkazů.

1.4 Výzkumné otázky

- O1: Které mechanismy agentního harnessu a vývojového životního cyklu umožňují agentovi samostatně provádět softwarovou změnu, zatímco rozhodnutí s vyšším dopadem zůstávají explicitně řízena člověkem?
- O2: Jak může agentní harness omezit nebo obnovit neproduktivní či přerušný běh bez ztráty již ověřeného stavu a bez opakování přijatých deterministických účinků?
- O3: Jak lze oddělit trvalý stav dlouhotrvající úlohy od omezeného kontextového okna

modelu tak, aby bylo možné práci po přerušení bezpečně obnovit a pokračovat v ní? O1 sleduje řízenou autonomii, O2 odolnost provádění a O3 kontinuitu stavu přes hranice jednotlivých modelových kontextů a běhů. Otázky jsou záměrně formulovány tak, aby na ně bylo možné odpovědět konkrétními architektonickými prvky a reprodukovatelnými důkazy z implementace.

1.5 Metodika práce

Práce používá konstrukční přístup odpovídající design science: vymezuje problém a cíle řešení, navrhuje a implementuje artefakt DarkFactory a následně jej vyhodnocuje pomocí reprodukovatelných technických důkazů. ^(4,5)

Teoretická část vychází z odborných článků, standardů, protokolových specifikací a primární dokumentace současných agentních systémů. Jednotlivé mechanismy jsou rozděleny do samostatných konceptů, aby bylo možné oddělit vlastnosti **Velký jazykový model (LLM)**, harnessu a technik Agentic Engineering.

Předmětem práce není trénování neuronových sítí, optimalizace vah ani matematický rozbor učení modelu. **Velký jazykový model (LLM)** je chápán jako hotová inferenční komponenta a je popsán pouze v rozsahu potřebném pro vysvětlení architektury okolního systému.

Praktická část používá zdrojový kód, typované kontrakty, testy, workflow a generované artefakty DarkFactory jako primární důkaz skutečné implementace. Evaluace rozlišuje architektonický důkaz, automatizované funkční testy, CI nad konkrétním commitem a provozní ověření na konkrétních cílových repozitářích. Tvrzení o kompletním produkčním Request lifecycle je přijato pouze tehdy, pokud existuje reprodukovatelný živý průchod od schválení Planningu po merge a rekonsiliaci; aktuální absence tohoto důkazu je uvedena jako omezení, nikoli nahrazena architektonickým předpokladem.

Práce neprovádí statistický benchmark obecné výkonnosti agentních systémů. Odpovědi na výzkumné otázky jsou omezeny na tvrzení podporovaná konkrétními reprodukovatelnými důkazy.

2 Teoretická část: Agentní vývoj softwaru

Vymezení konceptů potřebných pro pochopení současného agentního vývoje softwaru. Navazuje na argument práce: od AI-asistovaného vývoje přes model a inferenci k Harnessu a Agentickému inženýrství.

2.1 Úvod

Teoretická část rozpracovává technické předpoklady argumentu formulovaného v části 1.2.

Rozsah začíná změnou způsobu vývoje softwaru při delegování práce AI a končí metodami, kterými se nad schopnostmi Harnessu navrhuje agentní chování. Trénování modelů, optimalizace vah a obecná teorie strojového učení zůstávají mimo rozsah práce.

2.2 AI-asistovaný vývoj

2.2.1 Úvod

AI ve vývoji softwaru posouvá část práce od přímého psaní k zadávání, delegování a kontrole změn.

Tato část sleduje tři důsledky tohoto posunu: způsob formulace práce, proces řízení změny a nezávislé ověření její kvality.

2.2.2 Zadání a způsob práce

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽⁶⁾

Termín zavedl Andrej Karpathy v roce 2025 pro styl práce, ve kterém vývojář převážně přijímá a směřuje modelový výstup místo detailního čtení kódu. ^(6, 7)

Tweet Andreje Karpathyho z 2. února 2025, ve kterém popsal původní význam Vibe Coding.



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · 6.8M Views

Obrázek 1: Původní tweet Andreje Karpathyho o Vibe Coding. ^(6,8)

Vývoj řízený specifikací (Spec-Driven Development)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽⁹⁾

Specifikace odděluje požadované chování a omezení od konkrétní implementace a slouží jako společný referenční bod pro člověka i agenta. ⁽⁹⁾

Plánování (Planning)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽¹⁰⁾

Plán rozděluje práci na kontrolovatelné části a určuje, podle jakých podmínek lze posoudit jejich dokončení. Závislosti mezi částmi práce lze reprezentovat například pomocí DAG. ⁽¹⁰⁾

2.2.3 Řízení změny

Správa verzí (Version Control)

Systém pro zaznamenávání historie změn souborů a práci s oddělenými liniemi vývoje. ⁽¹¹⁾

Historie commitů umožňuje změny porovnávat, vracet a slučovat; větve umožňují oddělit souběžnou práci před integrací. ⁽¹¹⁾

Větev (Branch)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů. ⁽¹¹⁾

Větev umožňuje izolovat souběžnou změnu od cílové větve a později ji sloučit po kontrole nebo ověření. ^(11, 12)

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽¹³⁾

Pull request zpřístupňuje diff navržené změny a její stav před integrací do cílové větve. ⁽¹³⁾

2.2.4 Kvalita a ověřování

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽¹⁴⁾

V této práci označuje zejména AI-generovaný software, jehož objem nebo zdánlivá úplnost převyšují jeho ověřenou funkčnost a udržitelnost.

Průběžná integrace (CI)

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽¹⁵⁾

CI převádí část podmínek kvality do opakovatelných strojově vyhodnotitelných kontrol spouštěných nad změnami. ⁽¹⁵⁾

Integrační test (Integration Test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽¹⁰⁾
Integrační test zachycuje chyby vznikající ve vzájemném propojení částí systému, které izolované testování jednotlivých komponent nemusí odhalit. ⁽¹⁰⁾

2.2.5 Závěr

Delegování práce AI nezmenšuje potřebu softwarově-inženýrských kontrol; přesouvá jejich význam k explicitnímu zadání, sledovatelným změnám a strojově ověřitelné zpětné vazbě.

Tento požadavek vede k otázce, co samotný model skutečně provádí a které schopnosti musí dodat okolní systém.

2.3 Jazykový model a inference

2.3.1 Úvod

Jazykový model je v této práci chápán jako pravděpodobnostní komponenta, která při inferenci zpracovává omezený vstup a generuje další tokeny.

Oddělení modelu od inferenčního enginu umožňuje přesně určit, které vlastnosti patří modelové architektuře, které běhovému provedení inference a které už musí zajistit Harness.

2.3.2 Jazykový model

Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽¹⁶⁾

Při autoregresivním generování dekodér z dosavadní sekvence vytváří reprezentaci použitou k odhadu následujícího tokenu. ⁽¹⁶⁾

Velký jazykový model (LLM)

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽¹⁷⁾

Současné LLM typicky používají architekturu *Transformer*, která při autoregresivním generování odhaduje další token z předchozí sekvence. ⁽¹⁶⁾

Tokenizér (Tokenizer)

Komponenta, která převádí vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekodování. ⁽¹⁸⁾

Pravidla tokenizace určují slovník a segmentaci vstupu; subword metody, například Byte Pair Encoding, umožňují skládat text z jednotek menších než celé slovo. ⁽¹⁸⁾

Token

Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru. ⁽¹⁸⁾

Token nemusí odpovídat celému slovu; subword tokenizace může pracovat s částmi slov nebo jinými textovými fragmenty. ⁽¹⁸⁾ Identifikátor tokenu je před zpracováním neuronovou sítí převeden na vektorovou reprezentaci.

Vektorová reprezentace (Embedding)

Vícerozměrná vektorová reprezentace diskrétních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽¹⁹⁾

V jazykovém modelu embedding převádí identifikátory tokenů na spojité vektory zpracovatelné neuronovou sítí; vektorové reprezentace lze také porovnávat podle podobnosti. ⁽¹⁹⁾



Obrázek 2: Ilustrativní 2D projekce vztahu $\text{král} - \text{muž} + \text{žena} \approx \text{královna}$; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.

2.3.3 Inference

Inferenční engine (Inference Engine)

Běhová vrstva, která načte jazykový model a provádí jeho inferenci nad vstupními tokeny. ⁽²⁰⁾

Inferenční engine zajišťuje praktické provedení modelu, například plánování inferenčních požadavků, správu výpočetních prostředků a práci s mezipamětí během generování. ⁽²⁰⁾

Kontextové okno (Context Window)

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. ⁽²¹⁾ Do aktivního kontextu se společně vkládají instrukce, uživatelské vstupy, historie a výsledky nástrojů. Samotná nominální délka okna nezaručuje, že model všechny vložené informace využije stejně spolehlivě. ⁽²¹⁾

Mezipaměť klíčů a hodnot (KV Cache)

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. ⁽²²⁾

Při generování dalšího tokenu lze uložené klíče a hodnoty předchozí sekvence znovu použít místo jejich úplného přepočítání; paměťové nároky cache přitom rostou s délkou aktivní sekvence. ^(22, 23)

2.3.4 Limity inference

Degradace kontextu (Context Rot)

V této práci označuje Context Rot pokles spolehlivosti, s níž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. ⁽²¹⁾

Experimenty s dlouhým kontextem ukazují, že výkon může záviset na poloze relevantní informace a klesat, když je umístěna uvnitř dlouhého vstupu. ⁽²¹⁾

Divergence modelu

V této práci označuje postupné odchýlení pracovní reprezentace nebo generované trajektorie modelu od zamýšleného cíle či skutečného stavu.

Divergenci mohou podporovat dlouhý nebo zahlcený kontext, ztrátová kompakce a převzetí nepřesné mezirepresentace jako dalšího vstupu. Tento pojem zde zahrnuje i jev dříve označovaný jako sémantický posun. ^(21, 24)

2.3.5 Závěr

Model a inferenční engine společně vytvářejí výstup v mezích dostupného kontextu a běhových prostředků.

Nevlastní však trvalý agentní stav, účinky v externím prostředí ani dlouhodobou kontrolu úlohy. Tyto odpovědnosti začínají až ve vrstvě Harness.

2.4 Harness

2.4.1 Úvod

Harness je běhová vrstva kolem modelové inference, která drží stav, opakuje agentní smyčku a propojuje model s nástroji a prostředím. ⁽²⁾

Jeho odpovědností je kontinuita běhu a provedení účinků mimo model. Strategie, podle které se tyto schopnosti skládají do cíleného chování, patří až do Agentického inženýrství.

2.4.2 Smyčka a stav

Agentní smyčka (Agent Loop)

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽²⁵⁾

ReAct formalizuje střídání rozhodnutí, akce a pozorování výsledku; nové pozorování se stává vstupem dalšího kroku. ⁽²⁵⁾



Obrázek 3: Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu.

Agentní sezení (Session)

Persistovaná jednotka, která vymezuje jeden souvislý agentní běh a umožňuje jeho pozdější pokračování. ⁽²⁶⁾

Session je vlastníkem identity a hranice pokračujícího běhu; jeho historický průběh vlastní **Přepis (Transcript)** a aktuální pracovní skutečnosti **Stav (State)**. ^(2, 26)

Přepis (Transcript)

Uspořádaný historický záznam událostí vzniklých během **Agentní sezení (Session)**, například zpráv, akcí a výsledků nástrojů. ⁽²⁾

Transcript odpovídá na otázku, co se během běhu stalo. Je historickým záznamem, nikoli reprezentací právě platného pracovního stavu. ⁽²⁾

Stav (State)

Persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů běhu. ⁽²⁾

State odpovídá na otázku, co je pro další krok právě platné; na rozdíl od **Přepis (Transcript)** nemusí zachovávat úplnou historii předchozích událostí. ⁽²⁾

2.4.3 Nástroje a prostředí

Prostředí agenta (Agent Environment)

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky. ⁽²⁾

Změna souboru nebo spuštění procesu mění stav prostředí mimo modelový kontext. Přístup k těmto účinkům zprostředkovávají **Nástroje (Tools)** a jejich bezpečnostní hranice. ⁽²⁾

Nástroje (Tools)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽¹⁾

Harness zprostředkuje požadavek na nástroj, provede operaci v prostředí a vrátí její výsledek modelu jako další pozorování. ^(1,27)

Vyvolávání nástrojů (Tool Calling)

Mechanismus, kterým model vybere nástroj a předá jeho strukturované argumenty Harnessu místo přímého provedení operace. Harness popisuje dostupné nástroje modelu a po výběru nástroje validuje argumenty, provede příslušnou operaci a výsledek vrátí do dalšího kroku agentní smyčky. JSON Schema je jedním z prostředků popisu rozhraní nástroje, nikoli samostatnou kategorií agentního mechanismu.

Spouštění kódu (Code Execution)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽²⁸⁾

Při vývoji softwaru zpřístupňuje agentovi například testy, buildy, formátovače a diagnostické příkazy místo odhadování jejich výsledku. ⁽²⁸⁾

Izolované prostředí (Sandbox)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁾ Sandbox vytváří bezpečnostní hranici kolem nedůvěryhodných účinků; konkrétní realizace může používat například kontejner nebo microVM. ^(2,29)

2.4.4 Dovednosti a rozšíření

Dovednosti (Skills)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽³⁰⁾

Agent Skill je definován souborem `SKILL.md` a může odkazovat na doplňující skripty, reference nebo další zdroje načítané podle potřeby. ⁽³⁰⁾

Formát SKILL.md. Povinný definiční soubor Agent Skill. Podle specifikace obsahuje YAML frontmatter následovaný instrukcemi v Markdownu; povinnými poli frontmatteru jsou `name` a `description`. ⁽³⁰⁾

Následující úplný příklad používá povinná pole i několik volitelných polí povolených specifikací: ⁽³⁰⁾

```
---
name: repository-tests
description: Run, diagnose, and summarize the repository test suite. Use
```

```

when a software change must be validated before completion.
license: MIT
compatibility: Requires the repository's test runner to be available in
the runtime.
metadata:
  author: DarkFactory
  version: "1.0"
allowed-tools: Bash Read
---

# Repository tests

Use this skill when a change must be validated against the repository's
tests.

## Procedure

1. Detect the test command documented by the repository.
2. Run the smallest relevant test scope first.
3. Fix or report failures before continuing.
4. Run the complete required suite before completion.
5. Report the commands run and the final result.

## Failure reporting

For every unresolved failure, include the command, failing target, and
relevant output.

```

Specifikace dovoluje vedle `SKILL.md` také volitelné adresáře například pro skripty, referen-
ce a assety; ty se načítají pouze podle potřeby. ⁽³⁰⁾

Plugins (Plugins)

V této práci označují Plugins distribuovatelný mechanismus pro doplnění schopností a
běhového chování agentního prostředí. ⁽³¹⁾
Konkrétní platformy mohou do jednoho plugin balíčku zahrnout různé druhy rozšíření,
například skills, agenty, hooks nebo konfiguraci MCP serverů. ⁽³¹⁾ Taxonomie této práce ale
odděluje koncept Skills od Plugins a pod Plugins řadí Tools, Scripts, Hooks a MCP.

Skripty (Scripts)

Spustitelné soubory nebo posloupnosti příkazů používané k deterministickému prove-
dení opakovatelné operace. ⁽³¹⁾

Agentní rozšíření mohou skripty používat pro transformace, validace nebo jiné kroky, které je výhodnější provést programově než novým modelovým rozhodnutím. ^(30, 31)

Hooks

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽³²⁾

Hook může před nebo po vybrané události spustit deterministickou logiku, například validaci, příkaz nebo jinou automatizaci. ⁽³²⁾

MCP

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽³³⁾

MCP odděluje klientskou AI aplikaci od serverů poskytujících nástroje a další schopnosti, takže integrace lze implementovat mimo vlastní jádro harnessu. ^(33, 34)

2.4.5 Závěr

Harness mění izolované modelové volání na stavový systém schopný opakovaně pozorovat prostředí, volit akce a zachovávat průběh práce.

Sám ale neurčuje, jak má být kontext kurátorován, autonomie omezována nebo práce rozdělována mezi více agentů. To je úloha Agentického inženýrství.

2.5 Agentické inženýrství

2.5.1 Úvod

Agentické inženýrství v této práci označuje návrh způsobu, jakým se schopnosti Harnessu skládají, omezují a koordinují tak, aby agent cíleně plnil delší úlohu.

Vrstva se soustředí na tři otázky: jaké instrukce a informace model dostává, jak jsou jeho kroky omezeny a ukončovány a jak se práce koordinuje mezi více agenty nebo pracovními větvemi.

2.5.2 Instrukce a kontext

Promptové inženýrství (Prompt Engineering)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽³⁵⁾

Prompt ovlivňuje pravděpodobnostní chování modelu, ale sám nevynucuje technickou bezpečnostní nebo autorizační hranici; pravidla, která musí systém garantovat, patří do **Guardrail** nebo jiné běhové kontroly.

Systémový prompt (System Prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽³⁵⁾

Systémový prompt poskytuje stabilní instrukční kontext, ale sám o sobě není technickou izolační ani autorizační hranicí. ⁽³⁵⁾

Kontextové inženýrství (Context Engineering)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽³⁶⁾

Aktivní kontext může obsahovat instrukce, popisy dostupných nástrojů, externí data a vybranou historii interakce. Jeho obsah je nutné kurátorovat vzhledem k omezené kapacitě a nerovnoměrnému využití dlouhého kontextu. ^(21, 36)

Vkládání kontextu (Context Injection)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽³⁶⁾
Just-in-time přístup umožňuje mimo modelový kontext uchovávat odkazy nebo trvalý stav a potřebná data načíst nástrojem až během běhu. ⁽³⁶⁾

Kompakce kontextu (Context Compaction)

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽³⁶⁾
Kompakce uvolňuje kapacitu pro další běh, ale příliš agresivní komprese může odstranit detaily, které se později ukážou jako důležité. ^(24, 36)

RAG

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽³⁷⁾
RAG odděluje znalost uloženou v externím korpusu od parametrů modelu a umožňuje vybírat podklady podle aktuálního dotazu. ⁽³⁷⁾

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽³⁸⁾
Přímá prompt injection přichází v uživatelském vstupu; nepřímá injection je vložena do externích dat, například webové stránky, e-mailu, dokumentu, repozitáře nebo zdroje RAG. ⁽³⁹⁾ Typickým příkladem je životopis obsahující skrytou instrukci, která se pokusí ovlivnit následné hodnocení kandidáta modelem, přestože dokument měl sloužit pouze jako data. ⁽³⁸⁾ Důsledky proto závisejí také na oprávněních a nástrojích, které má agent k dispozici. ⁽⁴⁰⁾

2.5.3 Řízení agentního chování

Cílené smyčky (Goal Loops)

V této práci označují řídicí smyčky, které opakuji jednání podle explicitního cíle, pozorovaného výsledku a podmínky dalšího pokračování nebo ukončení.

Na rozdíl od samotného běhového Agent Loopu zahrnuje Goal Loop také zpětnou vazbu k dosažení cíle, ověření výsledku a rozhodnutí, zda pokračovat, změnit postup nebo běh ukončit. ^(1, 25)

Guardrail

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁴¹⁾ Agentní frameworky mohou guardrails implementovat různými způsoby; zde je důležitá deterministicky vyhodnotitelná hranice oddělená od samotného modelového rozhodnutí. Kontrola může běh zastavit nebo odmítnout konkrétní akci před provedením jejího účinku. ⁽⁴¹⁾

Člověk ve smyčce (HITL)

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁴²⁾ HITL ponechává vybraná rozhodnutí člověku místo úplné automatizace. V agentním workflow může být lidský zásah explicitní přechod nebo schvalovací bod, po kterém automatizované provádění pokračuje. ⁽⁴²⁾

2.5.4 Orchestrace agentů

Subagent

Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁴³⁾ Subagent umožňuje oddělit roli, instrukce a pracovní kontext dílčí úlohy od koordinujícího běhu a následně vrátit výsledek zpět nadřazené orchestrace. ^(43, 44)

Orchestrátor (Orchestrator)

Koordinační role nebo komponenta, která rozhoduje, kterému specializovanému agentovi předat dílčí práci a jak jeho výsledek začlenit do pokračujícího běhu. ⁽⁴³⁾ Centralizovaný orchestrátor zůstává vlastníkem hlavního workflow a může specialisty volat jako omezené pracovní jednotky, případně jejich práci kombinovat nebo spouštět paralelně. ^(1, 43)

Předání řízení (Handoff)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi. ⁽⁴³⁾ Handoff se liší od centralizované orchestrace tím, že specialista není pouze zavolán jako dílčí pracovní jednotka a vrácen orchestrátoru, ale přebírá aktivní pokračování interakce nebo úlohy. ⁽⁴³⁾

Pracovní graf (Workflow Graph)

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky.⁽⁴⁴⁾

Workflow Graph může vyjádřit sekvenční, podmíněné i paralelní větvení a může koordinovat více agentů. Pokud hrany vyjadřují pouze acyklické závislosti, může mít podobu DAG; workflow s návraty nebo opakováním však obecně DAG být nemusí.^(1,44)

2.5.5 Závěr

Agentické inženýrství spojuje návrh kontextu, řízení agentního chování a orchestrace do jedné vrstvy nad schopnostmi poskytovanými Harnessem. Praktická část tyto abstrakce znovu redefinuje; ukazuje jejich konkrétní realizaci a následně odděleně vyhodnocuje dostupné důkazy.

3 Praktická část

Aplikace teoretických principů na konkrétní systém DarkFactory. Praktická část propojuje teoretické vrstvy s jejich realizací v DarkFactory. Oddíl 3.2 je vyhrazen kanonické automaticky generované dokumentaci systému; ručně udržovaný text se soustředí na životní cyklus změny a na vyhodnocení dostupných důkazů.

3.1 Úvod

Vymezení způsobu, jakým praktická část aplikuje teoretické mechanismy na systém DarkFactory.

Praktická část popisuje pouze ověřenou implementaci DarkFactory a následně hodnotí chování a důkazy, které lze skutečně doložit.

3.2 DarkFactory

3.3 Životní cyklus změny

3.3.1 Úvod

Životní cyklus změny popisuje, jak DarkFactory převádí trvalé zadání na řízenou změnu repozitáře a jak mezi jednotlivými fázemi zachovává schválený záměr a pozorovatelný stav. Pořadí odděluje zadání a plán, provedení změny, její ověření a revizi, finalizaci a případné pokračování po přerušení. Jednotlivé mechanismy jsou zde popsány pouze v rozsahu specifickém pro tento tok; obecné pojmy z teoretické části se znovu nedefinují.

3.3.2 Zadání a plán

Řízená změna začíná trvalým Requestem a jeho převodem na konkrétní Planning artefakt před zahájením implementace. Request zachovává původní zadání a stav doručení. Planning jej spojuje s aktuálním stavem repozitáře, acceptance criteria, rozsahem, závislostmi, pořadím práce a očekávaným ověřením. Tato fáze vlastní pouze vznik schváleného rámce změny; review/fix implementace patří až do následného ověřování. ⁽⁴⁵⁾

Požadavek (Request)

Trvale evidovaná jednotka práce, která zachovává původní zadání a stav doručení před zahájením implementace. DarkFactory váže řízenou práci na Request reprezentovaný v GitHubu a používá jeho verzi, acceptance criteria a vztahy jako vstup do Planning a následného životního cyklu.

Plán DarkFactory (Planning)

Verzovaný a revidovaný plánovací artefakt, který převádí konkrétní Request a aktuální stav repozitáře na závazný rámec implementace a ověření.

Planning zachovává verbatim Request, acceptance criteria, behavioral contract, scope, exclusions, dependencies, sequencing, verification a známé či ještě zjišťované implementační vlastníky.

3.3.3 Implementace

Po schválení plánu systém provede požadovanou změnu nad pracovním prostředím repozitáře.

Model může navrhovat a provádět kroky prostřednictvím dostupných schopností, ale zdrojem pravdy zůstává pozorovaný stav pracovního stromu a skutečně provedené účinky. Tato část proto popisuje provedení změny, nikoli její následné hodnocení. ⁽⁴⁵⁾

3.3.4 Ověření a revize

Po implementaci se skutečný výsledek změny odvodí z pozorovatelných důkazů a samostatně se zkontroluje jeho správnost.

Deterministické ověření pracuje se změněnými soubory, rozsahem a výsledky verifikačních akcí. Review/Fix Loop zachycuje zjištění a případné opravy; každá materiální oprava musí znovu projít odpovídajícím ověřením. ⁽⁴⁵⁾

Deterministické ověření (Deterministic Verification)

Ověření výsledku změny pomocí přímo pozorovatelných strojových důkazů místo spoléhání na tvrzení modelu o tom, co provedl.

DarkFactory odvozuje výsledek kódového kroku ze skutečně změněných souborů, scope checku, detekovaných verifikačních akcí, jejich exit stavů a případně vytvořeného commitu.

Smyčka revize a opravy (Review/Fix Loop)

Opakovaný DarkFactory mechanismus, ve kterém je Planning nebo implementace nezávisle zkontrolována, zjištění jsou strukturovaně zachycena a následná oprava pokračuje do čistého výsledku.

Runtime ukládá předmět revize, fingerprint kontextu, nalezené problémy, stav čistoty, číslo iterace a historii review/fix kroků.

3.3.5 Finalizace

Před ukončením změny se implementace porovná se schváleným plánem a dosažený výsledek se promítne do trvalého stavu vývojového procesu.

Final Alignment kontroluje soulad hotové změny s aktuálním schváleným Planningem.

Reconciliation následně sjednocuje stav navázaného Requestu, pull requestu a dalších trvalých workflow objektů podle skutečného výsledku. ⁽⁴⁵⁾

Finální kontrola souladu (Final Alignment)

Závěrečná kontrola, která před integrací ověřuje soulad implementace s aktuálním schváleným Planningem a případnými schválenými změnami rozsahu.

DarkFactory provádí Final Alignment až po deterministickém ověření a čisté implementační review/fix smyčce; teprve poté mohou následovat povinné kontroly, finální review a merge.

Rekonciliace stavu (Reconciliation)

Sjednocení trvalého stavu Requestu, pull requestu, projektu a souvisejících workflow po dokončení nebo změně životního cyklu.

DarkFactory používá kanonické stavy Backlog, ToDo, In Progress, Blocked, Done, Superseded a Dropped a po terminálních událostech aktualizuje navázaný trvalý stav podle skutečně dosaženého výsledku.

3.3.6 Přerušeni a obnova

Přerušeni běhu nesmí samo o sobě způsobit opakování již potvrzených účinků ani ztrátu rozhodujícího pracovního stavu.

DarkFactory proto váže pokračování na persistovaný stav a kontrolu jeho původu a aktuálnosti. Detailní mechanismus Recovery bude vlastněn kanonickou autogenerated dokumentací DarkFactory v části 3.2; zde je uvedena pouze jeho role v životním cyklu změny.

(45)

3.3.7 Závěr

Životní cyklus odděluje záměr, provedení, důkaz správnosti a terminální stav změny tak, aby žádná z těchto vrstev nebyla nahrazena tvrzením modelu.

Tato separace poskytuje základ pro následující evaluaci: jednotlivé mechanismy lze ověřovat samostatně a zároveň lze sledovat, zda jejich kombinace funguje na úrovni celého systému.

3.4 Vyhodnocení

Vyhodnocení odděluje důkazy o jednotlivých mechanismech od důkazů o jejich spolupráci na úrovni systému a od přenosu na konkrétní cílové repozitáře.

Metodika těchto důkazů je stanovena v části 1.5; zde se už neopakuje. Tato část pouze uvádí pevný evidence snapshot, výsledky a hranice závěrů, které z nich lze odvodit.

3.4.1 Úvod

Vyhodnocení používá reprodukovatelný evidence snapshot DarkFactory na commitu `e9c10221b40589512d262a0edb95f709b923150c` a odpovídající CI run `35616745304`.^(46, 47) Snapshot slouží pouze jako pevný referenční bod pro výsledky. Kritéria a rozlišení architektonických, funkčních a integračních důkazů jsou vlastněna metodikou práce v části 1.5.

3.4.2 Ověření mechanismů

Referenční CI run dokončil všech 15 jobů úspěšně a hlavní Bun testovací sada vykázala 670 úspěšných testů ve 102 souborech bez selhání.⁽⁴⁷⁾

Samostatné testy ověřují mimo jiné persistenci Run State, detekci neaktuálního Planningu, zachování historie při provider failoveru, odvozování výsledku z pracovního stromu, kontrolu scope a recovery provenance.⁽⁴⁶⁾ Tato evidence podporuje konkrétní mechanismy; sama o sobě neprokazuje úspěšnost celého produkčního životního cyklu.

3.4.3 Ověření systému

Integrační testy dokazují spolupráci hlavních runtime mechanismů, ale evidence uzavřená 21. září 2026 neobsahuje jeden reprodukovatelný živý průchod celým produkčním životním cyklem změny.

Testy pokrývají workflow přechody, review/fix iterace, persistovaný stav, GitHub události, required-check gate, nástrojové účinky, failover a recovery.⁽⁴⁶⁾ Plný df-only průchod od schválení Planningu po merge a následnou rekonsiliaci zůstal zároveň otevřenou acceptance položkou Requestu 359, a proto není v této práci považován za prokázaný.⁽⁴⁸⁾

3.4.4 Ověření na repozitářích

Fleet-level ověření rozlišuje aktivní cílové repozitáře od historických nebo archivovaných položek a hodnotí pouze konkrétní commity s dohledatelnými workflow výsledky.

Repozitář `omnis` byl ověřen na commitu `a53660a1c0c6619f94768e5d520405052fb03df6`.

Jeho pipeline run `34708160162` dokončil úspěšně mimo jiné web, paper a docs jobs; samostatné deploy-docs a release joby na stejném commitu byly rovněž úspěšné.^(49,50)

Repozitář `ChessWithQuests` byl ověřen na commitu `50a50797f29c2a636d973981993191a65df3d131`. Pipeline run `34708180783` dokončil úspěšně paper, web a docs jobs a na stejném commitu uspěly také verify-docs, deploy-docs a release kontroly.^(51,52)

Původně uváděné repozitáře `template-OdbornaPrace` a `OdbornaPrace-mono` jsou v době evaluace archivované a nejsou proto považovány za aktivní fleet cíle.^(53,54)

Aktivním repozitářem práce je `DarkFactory-Paper`, který nahrazuje starší označení `OdbornaPrace-paper`. Snapshot `5bc04974f9aed0f55295389154124a09059ff35e` prošel úspěšně CI, Deploy Documentation i Release workflow.^(55–58)

Aktivní fleet evidence tedy v této fázi pokrývá DarkFactory, omnis, ChessWithQuests a DarkFactory-Paper. Dvě původní fleet položky jsou archivované; úplné 361 fleet acceptance proto nelze z těchto výsledků považovat za uzavřené.

3.4.5 Výzkumné otázky

Odpovědi jsou omezeny na vlastnosti doložené implementací, automatickými testy, CI a dostupnými provozními artefakty.

01. Řízenou autonomii podporuje oddělení schváleného plánu, implementace, deterministického ověření, review/fix smyčky, Final Alignment a merge autorizace. Jejich dílčí kontrakty jsou implementované a testované, ale evidence neobsahuje jeden úplný živý produkční průchod. ^(46–48)

02. Přerušný nebo neproduktivní běh lze omezovat pomocí budgetů, klasifikace chyb a failoveru a obnovovat nad persistovaným stavem s kontrolou provenance. Testy tyto mechanismy podporují, nikoli však živou crash/resume idempotenci nad celým produkčním workflow. ^(46, 48)

03. Trvalý pracovní stav je oddělen od context window: Run State, Session a Transcript mohou existovat mimo aktuální inferenční vstup a aktivní kontext z nich vybírá pouze informace potřebné pro další krok. Tato architektura podporuje pokračování přes hranice modelových běhů, ale nedokazuje nulovou informační ztrátu pro libovolně dlouhou úlohu. ⁽⁴⁶⁾

3.4.6 Diskuse a omezení

Dostupná evidence podporuje realizaci a testované vlastnosti jednotlivých mechanismů, nikoli obecné tvrzení o úplné autonomii nebo výkonnostní převaze DarkFactory. Práce neprovádí statistický benchmark úspěšnosti, ceny, latence ani četnosti zacyklení na reprezentativním souboru úloh. Evidence také neobsahuje jeden živý plný Request lifecycle ani úplnou acceptance původně plánované fleety. ⁽⁴⁸⁾ Silnou stránkou evaluace je naopak přímá vazba konkrétních tvrzení na implementační a testovací důkazy, například pozorovaný stav pracovního stromu nebo recovery provenance. ⁽⁴⁶⁾

3.4.7 Závěr

Evaluace prokazuje implementaci a automatické ověření klíčových mechanismů DarkFactory a současně přesně vymezuje chybějící systémové důkazy. Hlavní cíl práce je proto hodnocen na úrovni navržené architektury a reprodukovatelně testovaných mechanismů. Neprovedený plný živý lifecycle není nahrazen silnějším tvrzením; zůstává explicitním omezením závěrů. ⁽⁴⁸⁾

4 Závěr

Práce ukazuje, že praktické použití agentní AI při vývoji softwaru nelze redukovat na samotné generování kódu jazykovým modelem; rozhodující část systému tvoří harness, který spravuje stav, prostředí, nástroje, ověřování a řídicí hranice.

Na systému DarkFactory byla tato architektura rozložena do explicitních runtime, capability, GitHub, identity a operátorských hranic. Referenční commit DarkFactory prošel automatickou quality pipeline a hlavní testovací sada vykazala 670 úspěšných testů bez selhání; samostatné testy ověřují mimo jiné persistenci Run State, Planning review, modelový failover, deterministické zachycení výsledku, recovery provenance a oddělení browserové autentizace od strojových credentials. ^(46, 47)

Výsledky podporují architektonickou odpověď na výzkumné otázky: řízená autonomie vyžaduje explicitní schvalovací a verifikační body, odolnost běhu vyžaduje persistovaný stav a kontrolovaný failover/recovery a kontinuita dlouhotrvající úlohy nemůže být závislá pouze na modelovém context window.

V evidence setu uzavřeném 21. září 2026 nebyl prokázán jeden živý df-only průchod celým Request lifecycle od schválení Planningu po merge a rekonciliaci; stejná podmínka zůstala otevřenou acceptance položkou Requestu 359 ⁽⁴⁸⁾ Práce proto nevyvozuje obecnou výkonnostní převahu DarkFactory ani úplnou autonomii; tato tvrzení zůstávají mimo rozsah provedené evaluace.

Seznam obrázků a tabulek

Obrázek 1	Původní tweet Andreje Karpathyho o Vibe Coding. ^(6,8)	14
Obrázek 2	Ilustrativní 2D projekce vztahu král – muž + žena $\approx$ královna; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslov- né naučené dimenze embeddingového prostoru.	17
Obrázek 3	Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu. .	19

Seznam příloh

A Encyklopedie a rejstřík pojmů	34
---------------------------------------	----

A Encyklopedie a rejstřík pojmů

Abecední přehled klíčových pojmů použitých v práci. Názvy a definice jsou odvozeny přímo z kanonických sémantických položek.

Agentní sezení (Session)

Persistovaná jednotka, která vymezuje jeden souvislý agentní běh a umožňuje jeho pozdější pokračování. ⁽²⁶⁾

Agentní smyčka (Agent Loop)

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽²⁵⁾

Cílené smyčky (Goal Loops)

V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorování výsledku a podmínky dalšího pokračování nebo ukončení.

Degradace kontextu (Context Rot)

V této práci označuje Context Rot pokles spolehlivosti, s níž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. ⁽²¹⁾

Deterministické ověření (Deterministic Verification)

Ověření výsledku změny pomocí přímo pozorovatelných strojových důkazů místo spoléhání na tvrzení modelu o tom, co provedl.

Dovednosti (Skills)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽³⁰⁾

Finální kontrola souladu (Final Alignment)

Závěrečná kontrola, která před integrací ověřuje soulad implementace s aktuálním schváleným Planningem a případnými schválenými změnami rozsahu.

Guardrail

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁴¹⁾

Harness

Harness je běhová vrstva kolem modelové inference, která drží stav, opakuje agentní smyčku a propojuje model s nástroji a prostředím. ⁽²⁾

Hooks

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽³²⁾

Inferenční engine (Inference Engine)

Běhová vrstva, která načte jazykový model a provádí jeho inferenci nad vstupními tokeny. ⁽²⁰⁾

Integrační test (Integration Test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽¹⁰⁾

Izolované prostředí (Sandbox)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁾

Kompakce kontextu (Context Compaction)

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽³⁶⁾

Kontextové inženýrství (Context Engineering)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽³⁶⁾

Kontextové okno (Context Window)

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. ⁽²¹⁾

MCP

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽³³⁾

Mezipaměť klíčů a hodnot (KV Cache)

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. ⁽²²⁾

Nástroje (Tools)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽¹⁾

Orchestrátor (Orchestrator)

Koordinační role nebo komponenta, která rozhoduje, kterému specializovanému agentovi předat dílčí práci a jak jeho výsledek začlenit do pokračujícího běhu. ⁽⁴³⁾

Pluginy (Plugins)

V této práci označují Plugins distribuovatelný mechanismus pro doplnění schopností a běhového chování agentního prostředí. ⁽³¹⁾

Plán DarkFactory (Planning)

Verzovaný a revidovaný plánovací artefakt, který převádí konkrétní Request a aktuální stav repozitáře na závazný rámec implementace a ověření.

Plánování (Planning)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽¹⁰⁾

Požadavek (Request)

Trvale evidovaná jednotka práce, která zachovává původní zadání a stav doručení před zahájením implementace.

Pracovní graf (Workflow Graph)

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. ⁽⁴⁴⁾

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽³⁸⁾

Promptové inženýrství (Prompt Engineering)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽³⁵⁾

Prostředí agenta (Agent Environment)

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky. ⁽²⁾

Průběžná integrace (CI)

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽¹⁵⁾

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽¹³⁾

Předání řízení (Handoff)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi. ⁽⁴³⁾

Přepis (Transcript)

Uspořádaný historický záznam událostí vzniklých během **Agentní sezení (Session)**, například zpráv, akcí a výsledků nástrojů. ⁽²⁾

RAG

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽³⁷⁾

Rekonciliace stavu (Reconciliation)

Sjednocení trvalého stavu Requestu, pull requestu, projektu a souvisejících workflow po dokončení nebo změně životního cyklu.

Skripty (Scripts)

Spustitelné soubory nebo posloupnosti příkazů používané k deterministickému provedení opakovatelné operace. ⁽³¹⁾

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽¹⁴⁾

Smyčka revize a opravy (Review/Fix Loop)

Opakovaný DarkFactory mechanismus, ve kterém je Planning nebo implementace nezávisle zkontrolována, zjištění jsou strukturovaně zachycena a následná oprava pokračuje do čistého výsledku.

Spouštění kódu (Code Execution)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽²⁸⁾

Správa verzí (Version Control)

Systém pro zaznamenávání historie změn souborů a práci s oddělenými liniemi vývoje. ⁽¹¹⁾

Stav (State)

Persistovaná reprezentace aktuálně platných pracovních skutečností a řídicích údajů běhu. ⁽²⁾

Subagent

Speciální agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁴³⁾

Systémový prompt (System Prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽³⁵⁾

Token

Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru. ⁽¹⁸⁾

Tokenizér (Tokenizer)

Komponenta, která převádí vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekodování. ⁽¹⁸⁾

Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽¹⁶⁾

Vektorová reprezentace (Embedding)

Vícerozměrná vektorová reprezentace diskrétních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽¹⁹⁾

Velký jazykový model (LLM)

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽¹⁷⁾

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽⁶⁾

Vkládání kontextu (Context Injection)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽³⁶⁾

Vyvolávání nástrojů (Tool Calling)

Mechanismus, kterým model vybere nástroj a předá jeho strukturované argumenty Harne-ssu místo přímého provedení operace.

Vývoj řízený specifikací (Spec-Driven Development)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽⁹⁾

Větev (Branch)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů.

⁽¹¹⁾

Člověk ve smyčce (HITL)

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁴²⁾

Seznam zdrojů

1. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
2. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>
3. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion Q1 2026: Trends and Insights. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
4. HEVNER, Alan R., MARCH, Salvatore T., PARK, Jinsoo a RAM, Sudha. Design Science in Information Systems Research. *MIS Quarterly*. 2004. Vol. 28, no. 1.
5. PEFFERS, Ken, TUUNANEN, Tuure, ROTHENBERGER, Marcus A. a CHATTERJEE, Samir. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*. 2007. Vol. 24, no. 3, p. 45–77. DOI [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
6. KARPATY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
7. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
8. KRAVETS, David. A semantic history: How the term 'vibe coding' went from a tweet to prod. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.coderabbit.ai/blog/a-semantic-history-how-the-term-vibe-coding-went-from-a-tweet-to-prod>
9. BÖCKELER, Birgitta. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. Online. 2025. [Accessed 21 září 2026]. Available from: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
10. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
11. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
12. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>
13. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
14. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>

15. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
16. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
17. BROWN, Tom B., MANN, Benjamin, RYDER, Nick, SUBBIAH, Melanie, KAPLAN, Jared, DHARIWAL, Prafulla, NEELAKANTAN, Arvind, SHYAM, Pranav, SASTRY, Girish, ASKELL, Amanda, HERBERT-VOSS, Ariel, KRUEGER, Gretchen, HENIGHAN, Tom, CHILD, Rewon, RAMESH, Aditya, ZIEGLER, Daniel M., WU, Jeffrey, WINTER, Clemens, HESSE, Christopher, CHEN, Mark, SIGLER, Eric, LITWIN, Mateusz, GRAY, Scott, CHES, Benjamin, CLARK, Jack, BERNER, Christopher, MCCANDLISH, Sam, RADFORD, Alec, SUTSKEVER, Ilya a AMODEI, Dario. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33. Available from: <https://arxiv.org/abs/2005.14165>
18. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
19. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
20. VLLM PROJECT. vLLM: A High-Throughput and Memory-Efficient Inference Engine for LLMs. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.vllm.ai/en/stable/api/vllm/>
21. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
22. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
23. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural*

- Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
24. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
 25. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
 26. OPENAI. Sessions | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sessions/>
 27. SCHICK, Timo, DWIVEDI-YU, Jane, DESS\I, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
 28. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>
 29. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
 30. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
 31. ANTHROPIC. Create plugins | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/plugins>
 32. ANTHROPIC. Hooks reference | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/hooks>
 33. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
 34. ANTHROPIC. Connect Claude Code to tools via MCP. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/mcp>
 35. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

36. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
37. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
38. OWASP GENAI SECURITY PROJECT. LLM01:2025 Prompt Injection. Online. 2025. [Accessed 21 září 2026]. Available from: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
39. OWASP FOUNDATION. LLM Prompt Injection Prevention Cheat Sheet. Online. 2026. [Accessed 21 září 2026]. Available from: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
40. OPENAI. Understanding prompt injections. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.com/safety/prompt-injections/>
41. OPENAI. Guardrails | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/guardrails/>
42. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
43. OPENAI. Agent orchestration | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: https://openai.github.io/openai-agents-python/multi_agent/
44. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
45. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
46. MARIUS, Patrik. DarkFactory commit e9c10221. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>

47. GITHUB ACTIONS. DarkFactory CI run 35616745304. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/actions/runs/35616745304>
48. MARIUS, Patrik. Request #359: finish the df production engine and retire Python orchestration. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/issues/359>
49. MARIUS, Patrik. omnis commit a53660a1. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/commit/a53660a1c0c6619f94768e5d520405052fb03df6>
50. GITHUB ACTIONS. omnis pipeline run 34708160162. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/actions/runs/34708160162>
51. MARIUS, Patrik. ChessWithQuests commit 50a50797. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/commit/50a50797f29c2a636d973981993191a65df3d131>
52. GITHUB ACTIONS. ChessWithQuests pipeline run 34708180783. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/actions/runs/34708180783>
53. MARIUS, Patrik. template-OdbornaPrace repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/template-OdbornaPrace>
54. MARIUS, Patrik. OdbornaPrace-mono repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/OdbornaPrace-mono>
55. MARIUS, Patrik. DarkFactory-Paper commit 5bc04974. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/commit/5bc04974f9aed0f55295389154124a09059ff35e>
56. GITHUB ACTIONS. DarkFactory-Paper CI run 35617820423. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820423>
57. GITHUB ACTIONS. DarkFactory-Paper Deploy Documentation run 35617820271. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820271>
58. GITHUB ACTIONS. DarkFactory-Paper Release run 35617820286. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820286>
59. OPENAI. Introducing Structured Outputs in the API. Online. 2024. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>

60. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>